

# **NCCL Collective Ops and Usage in DPP and FSDP**

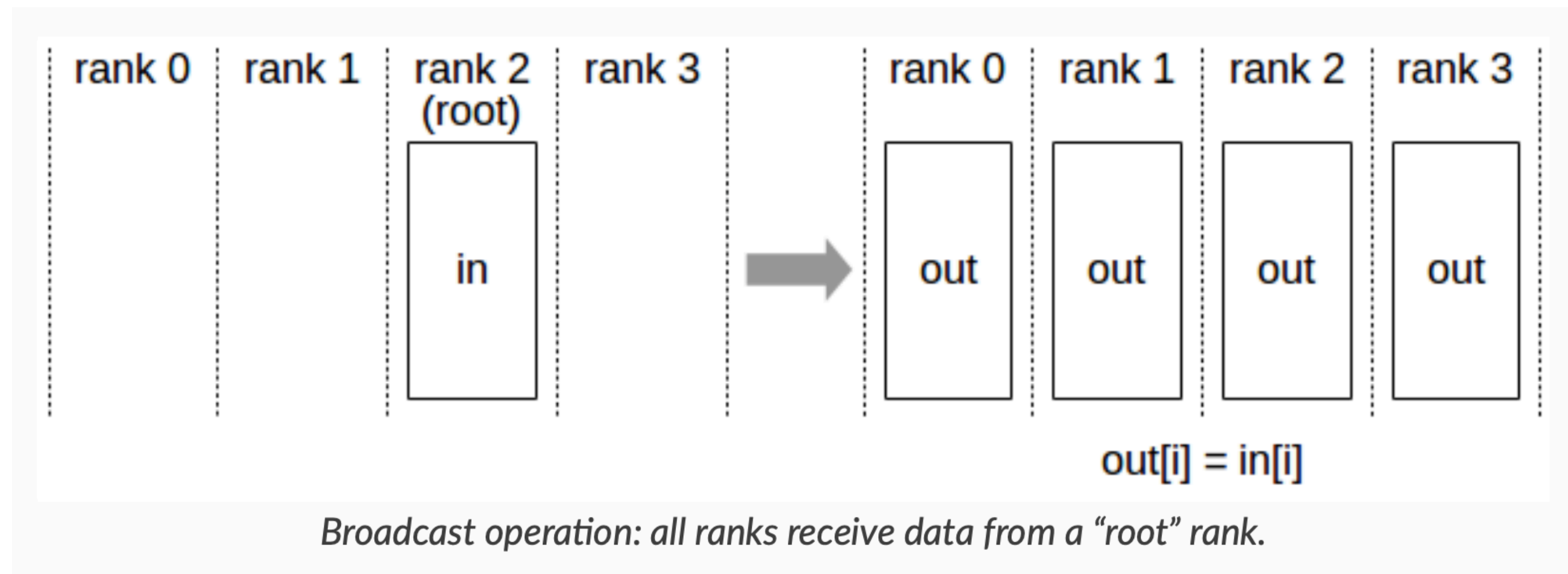
# Collective Operations

| Type         | Operations     | Application                                                        |
|--------------|----------------|--------------------------------------------------------------------|
| One to Many  | Broadcast      | Model Initialization                                               |
| Many to one  | Reduce         | Evaluation/logging on a Single GPU                                 |
| Many to Many | Reduce-Scatter | FSDP: after backward pass, average and shard gradients to each GPU |
|              | AllGather      | FSDP: before forward and backward, get full copy of model          |
|              | All-Reduce     | DP: aggregate gradients                                            |

**ALLReduce = ReduceScatter + AllGather**  
**AllReduce = Reduce + broadcast**

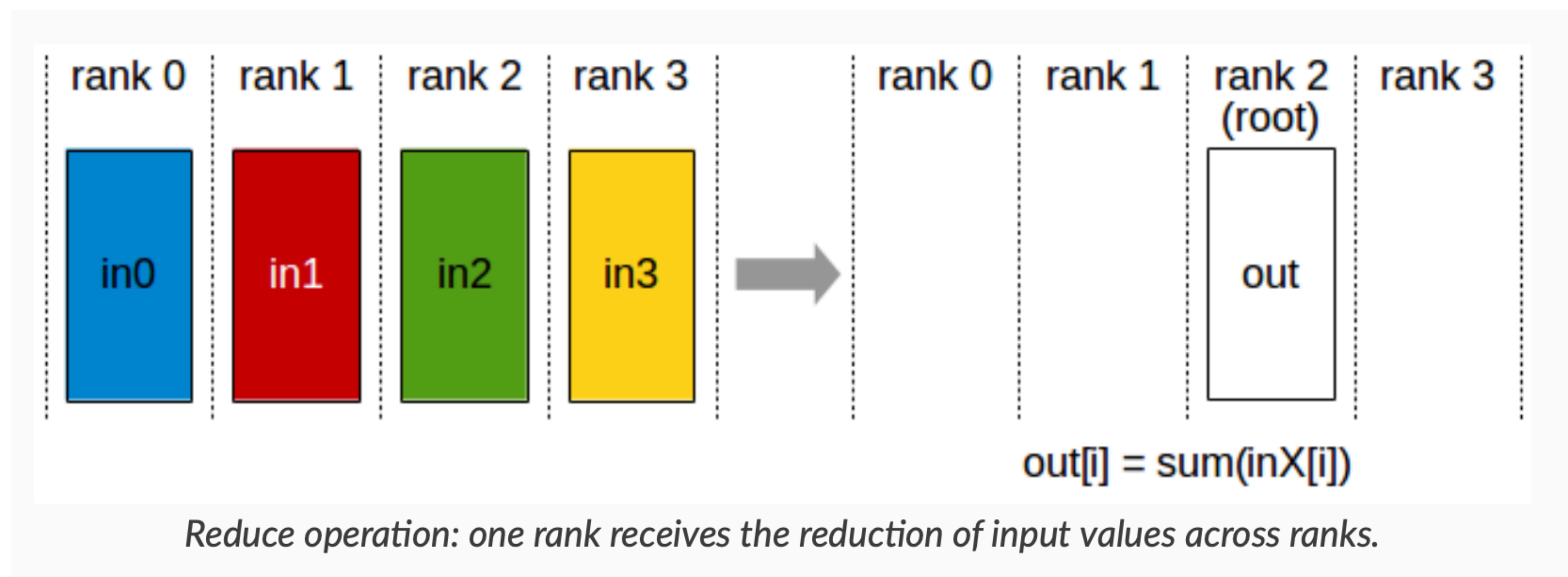
# Collective Operations

## Broadcast



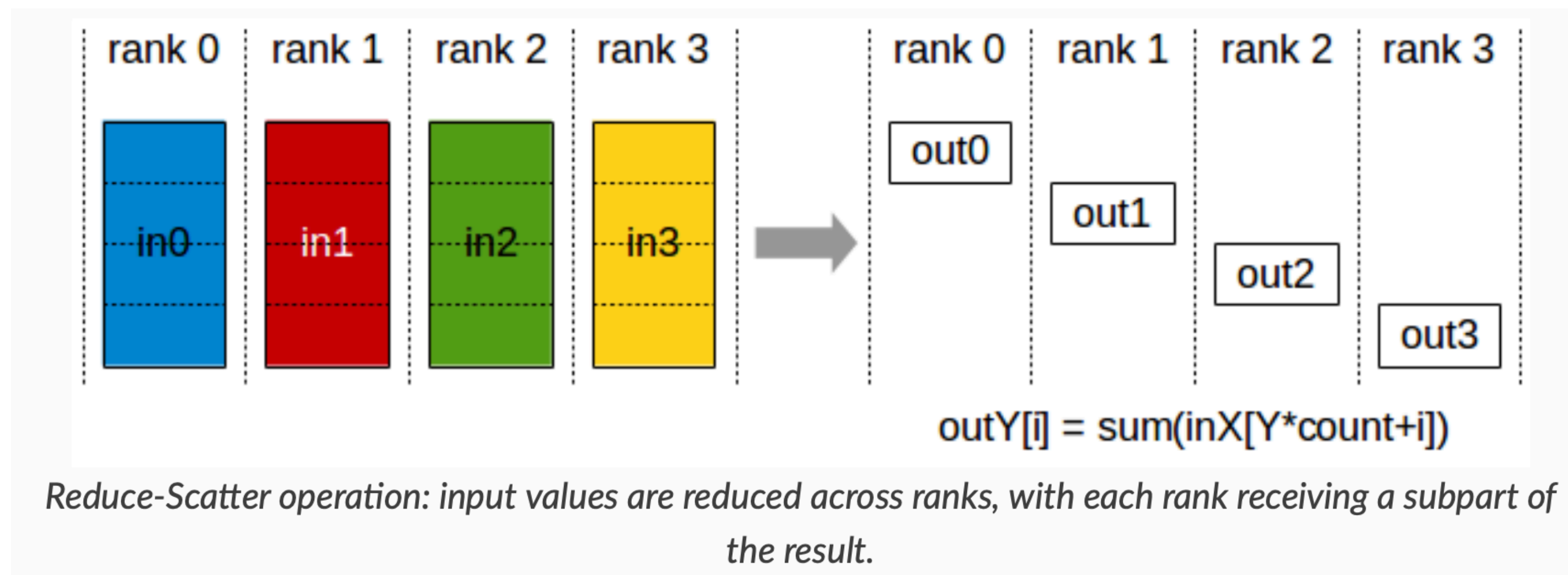
# Collective Operations

## Reduce



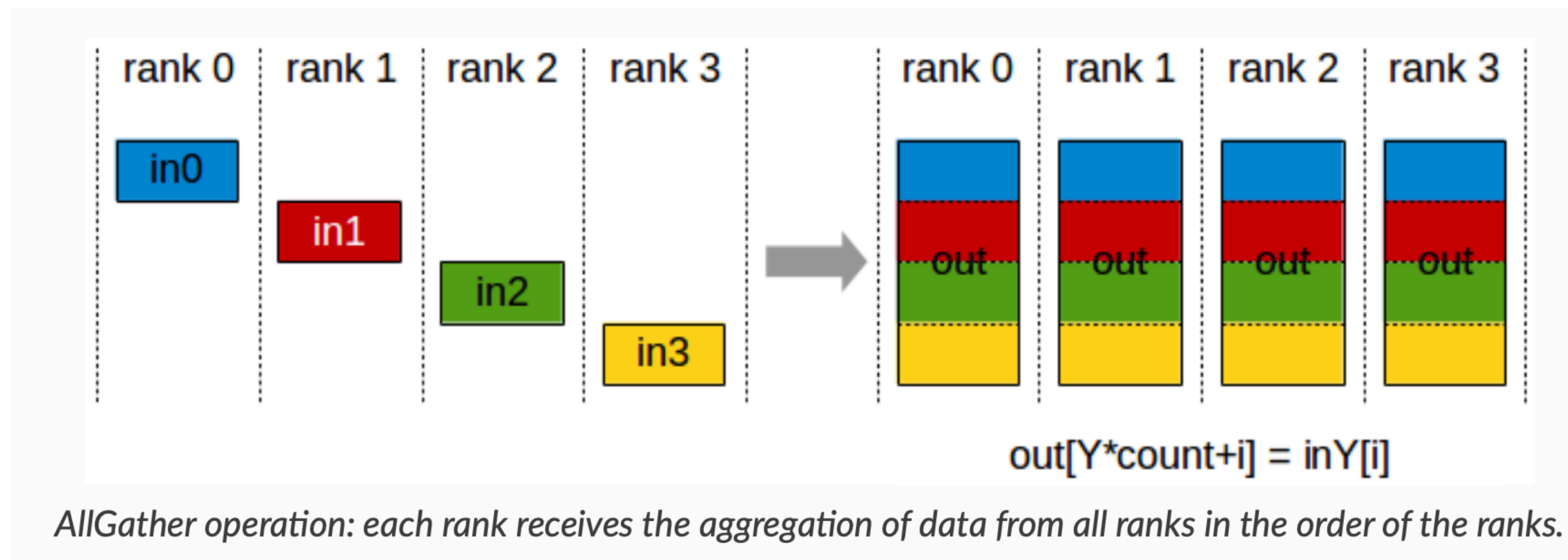
# Collective Operations

## Reduce-Scatter



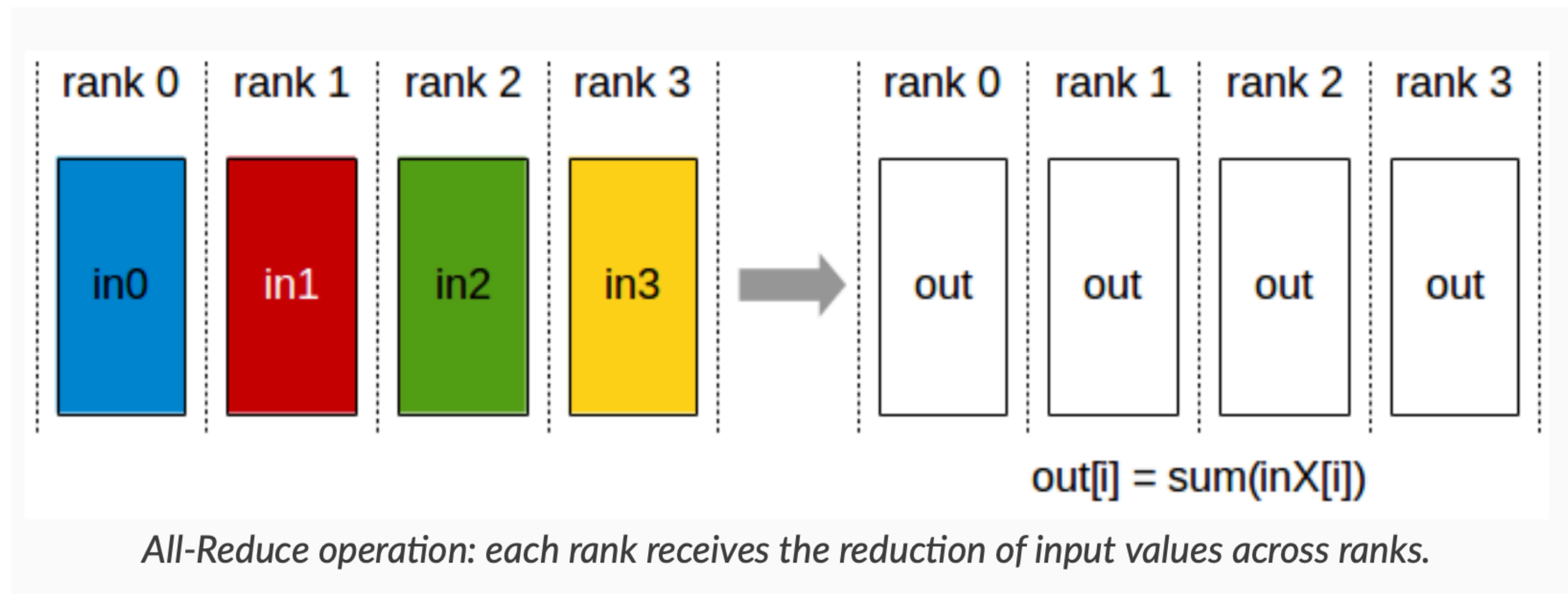
# Collective Operations

## AllGather

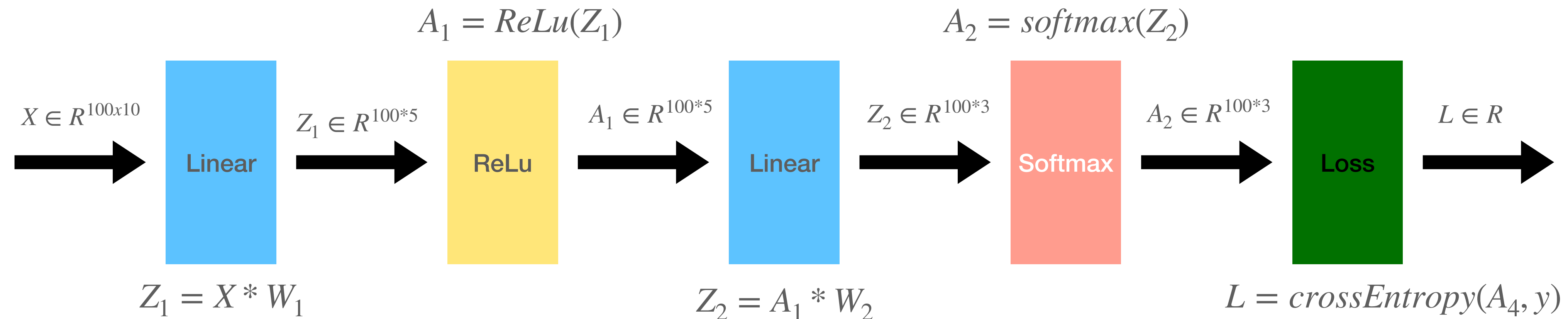


# Collective Operations

## All-Reduce



# Two layers Network



## Input

$$X \in R^{100 \times 10}$$

$$y \in R^{100}$$

## Parameters

$$W_1 \in R^{10 \times 5}$$

$$W_2 \in R^{5 \times 3}$$

## Forward

$$Z_1 = X * W_1$$

$$A_1 = ReLu(Z_1)$$

$$Z_2 = A_1 * W_2$$

$$A_2 = softmax(Z_2)$$

$$L = crossEntropy(A_2, y)$$

## Backward

$$dZ_2 = Z_2 - one\_hot(y)$$

$$dW_2 = \frac{A_1^T * dZ_2}{m}$$

$$dA_1 = dZ_2 * W_2^T$$

$$dZ_1 = dA_1 * Relu\_derivative(A_1)$$

$$dW_1 = \frac{X^T * dZ_1}{m}$$

## Update

$$W_2 = W_2 - \eta dW_2$$

$$W_1 = W_1 - \eta dW_1$$

Have to have full copy of weights to backward.

# Two layers Network

## Input

$$X \in R^{100 \times 10}$$

$$y \in R^{100}$$

## Parameters

$$W_1 \in R^{10 \times 5}$$

$$W_2 \in R^{5 \times 3}$$

## Forward

$$Z_1 = X * W_1$$

$$A_1 = ReLu(Z_1)$$

$$Z_2 = A_1 * W_2$$

$$A_2 = softmax(Z_2)$$

$$L = crossEntropy(A_2, y)$$

## Backward

$$dZ_2 = Z_2 - one\_hot(y)$$

$$dW_2 = \frac{A_1^T * dZ_2}{m}$$

$$dA_1 = dZ_2 * W_2^T$$

$$dZ_1 = dA_1 * Relu\_derivative(A_1)$$

$$dW_1 = \frac{X^T * dZ_1}{m}$$

## Update

$$W_2 = W_2 - \eta dW_2$$

$$W_1 = W_1 - \eta dW_1$$

Have to have full copy of weights to backward.

---

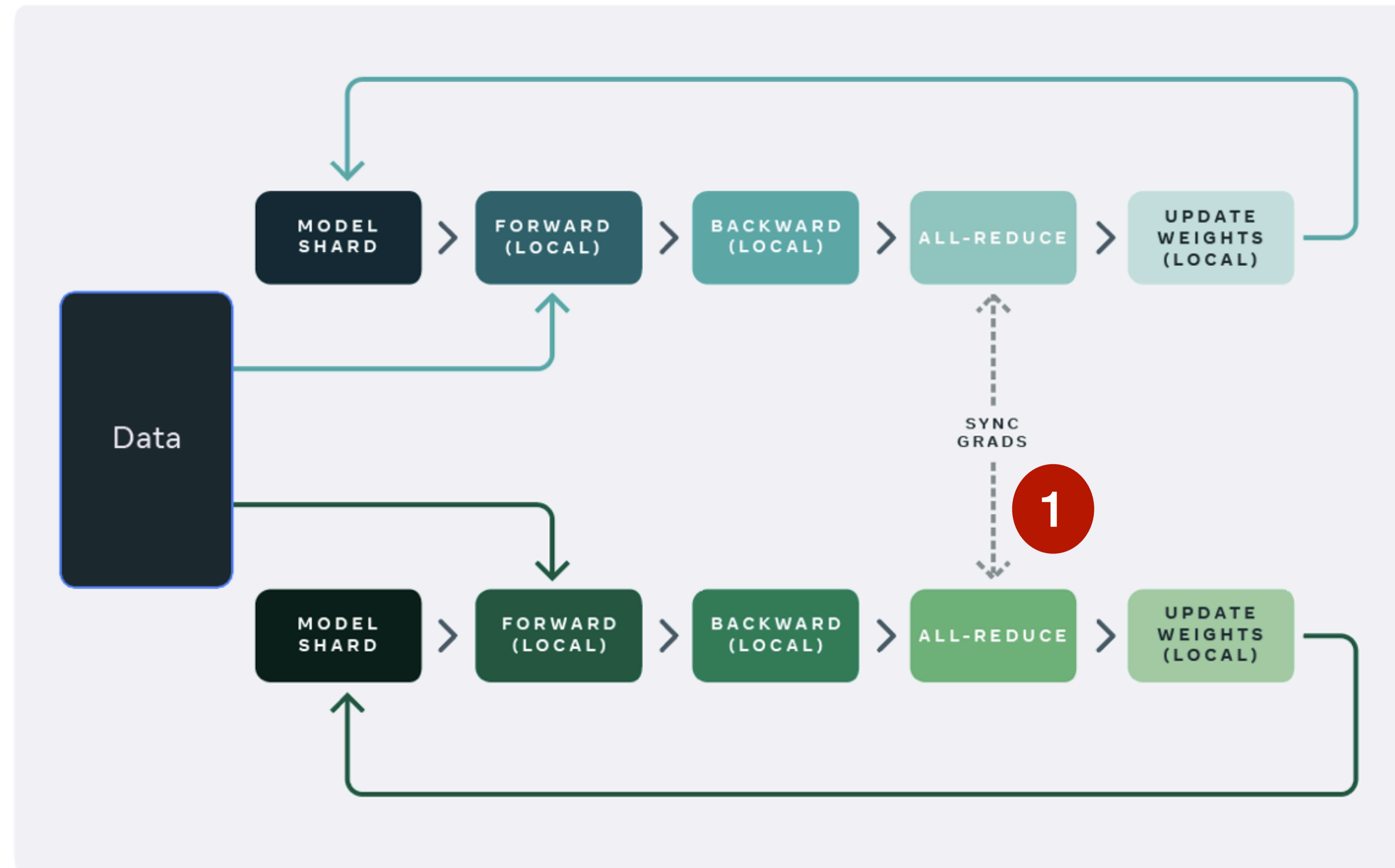
PyTorch

Loss = model(input)

loss.backward()

optimizer.step()

# DDP

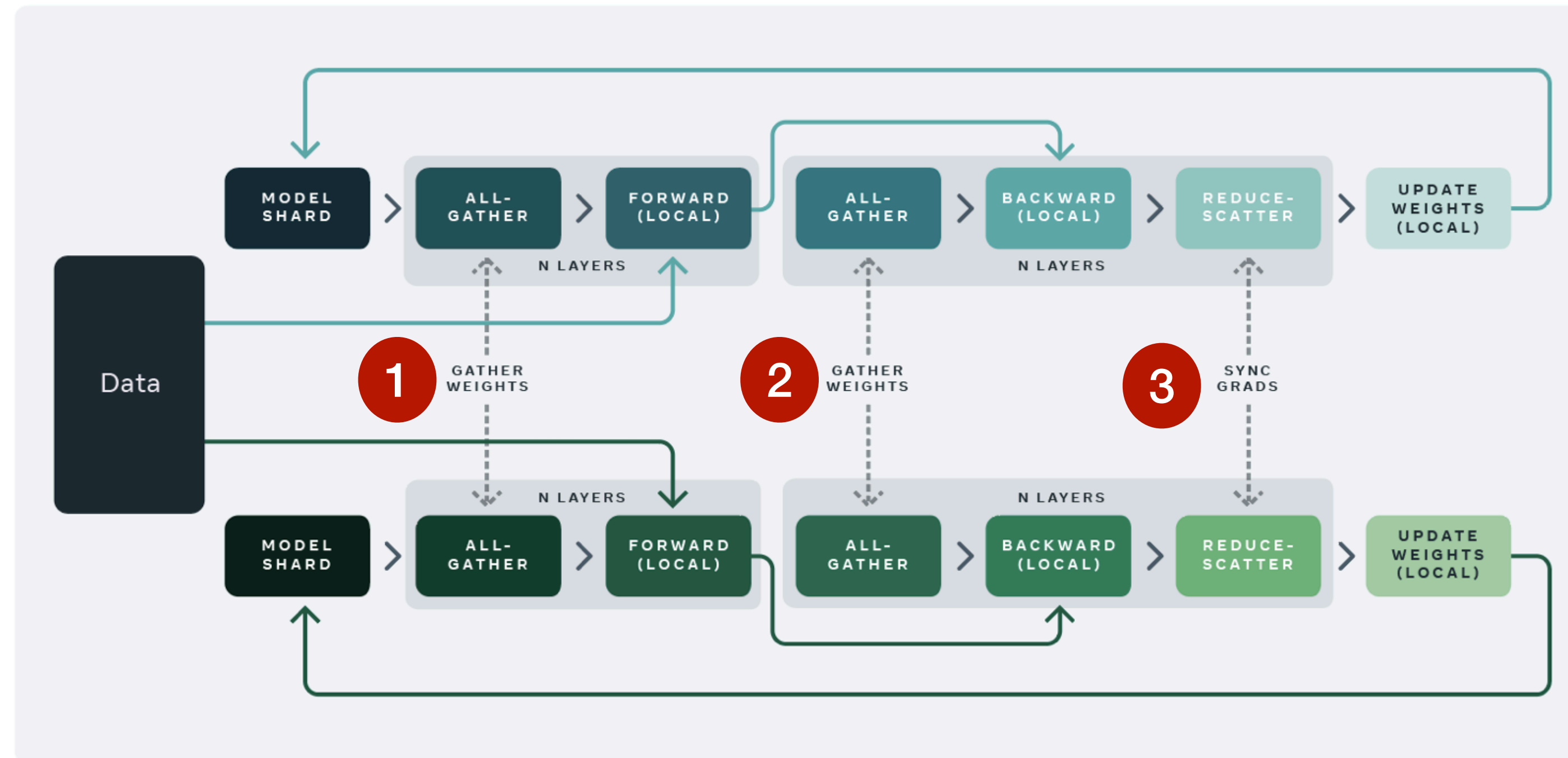


1

All-reduce: average gradients from each GPU, then locally update weights.

<https://engineering.fb.com/2021/07/15/open-source/fsdp/>

# FSDP



- 1 All-Gather: to get full copy of model from each GPU and do forward pass locally.
- 2 All-Gather: to get full copy of model from each GPU before backward pass for computing gradients.
- 3 Reduce-scatter: to average gradients and shard gradients.

Why?